Lecture 03: Contiguous Tables + Aggregation Pipelines

Goals

- Model data as tables (rows + columns)
- Build extraction -> normalization -> aggregation pipelines
- Avoid misaligned columns in parallel arrays

Data recap (Spotify)

Demo files live in `lectures/data/` (or `data/` if you open this notebook inside `lectures/`). We will reformat list-of-dicts into table form for faster scans and summaries.

Segment 1: Table mindset

Goal: rows + columns with a header.

In [ ]:

```python
from pathlib import Path
import csv

data_dir = Path('lectures/data')
if not data_dir.exists():
    data_dir = Path('data')

def parse_int(value, default=0):
    try:
        return int(value)
    except (TypeError, ValueError):
        return default

def clean_track_row(row):
    return {
        'track_id': row['track_id'].strip().casefold(),
        'track_name': row['track_name'].strip(),
        'artist_id': row['artist_id'].strip().casefold(),
        'genre': (row['genre'] or 'unknown').strip().casefold(),
        'duration_ms': parse_int(row['duration_ms']),
    }

with (data_dir / 'tracks.csv').open(newline='', encoding='utf-8') as f:
    reader = csv.DictReader(f)
    tracks = [clean_track_row(r) for r in reader]

header = ['track_id', 'track_name', 'artist_id', 'genre', 'duration_ms']
table = [header] + [[t[col] for col in header] for t in tracks]
table[:4]
```

In [ ]:

```
def column(table, name):
    idx = table[0].index(name)
    return [row[idx] for row in table[1:]]

column(table, 'genre')
```

Segment 2: Parallel arrays

Goal: keep aligned lists for fast scans.

In [ ]:

```
track_ids = column(table, 'track_id')
genres = column(table, 'genre')
durations = column(table, 'duration_ms')

list(zip(track_ids, genres, durations))
```

Alignment check

- All arrays must have the same length
- The same index points to the same track

Segment 3: Pipeline stages

Goal: extract -> normalize -> aggregate -> report.

In [ ]:

```
def extract_tracks(rows):
    return [[r['track_id'], r['artist_id'], r['genre'], r['duration_ms']

def normalize_rows(rows):
    return [[tid.strip().casefold(), aid.strip().casefold(), (g or 'unk
            for tid, aid, g, dur in rows]

def report_top_genres(rows, k=3):
    from collections import Counter
    counts = Counter(r[2] for r in rows)
    return counts.most_common(k)

stage1 = extract_tracks(tracks)
stage2 = normalize_rows(stage1)
report_top_genres(stage2)
```

Segment 1-3 summary

- Tables make scans predictable
- Parallel arrays are fast but easy to misalign
- Pipelines keep each step focused

In-class exercise 1 (commit)

Build a `table` with a header row for plays (`play_id`, `track_id`, `play_count`).

- Write a `column_sum(table, 'play_count')` helper
- Commit your solution

Break (3 minutes)

Segment 4: Artist-by-genre table

Goal: produce a 2D summary table.

In [ ]:

```
def clean_play_row(row):
    return [
        row['play_id'].strip().casefold(),
        row['track_id'].strip().casefold(),
        parse_int(row['play_count']),
    ]

with (data_dir / 'plays.csv').open(newline='', encoding='utf-8') as f:
    reader = csv.DictReader(f)
    plays = [clean_play_row(r) for r in reader]

plays[:4]
```

In [ ]:

```
# Convert summary dict to a table with header
summary_table = [['artist_id', 'genre', 'total_plays']]
for (artist_id, genre), total in summary.items():
    summary_table.append([artist_id, genre, total])

summary_table
```

Segment 5: Verify table shape

Goal: catch misaligned columns early.

In [ ]:

```
def verify_table(table):
    width = len(table[0])
    return all(len(row) == width for row in table)

verify_table(summary_table)
```

Segment 6: Tradeoffs

Goal: know when tables beat dicts.

- Tables are great for scans, sorting, and exporting
- Dicts are great for lookups and sparse data
- Choose the shape that matches your dominant operation

Segment 4-6 summary

- Build summary tables from dict aggregations
- Verify shapes to avoid silent errors
- Tables and dicts complement each other

In-class exercise 2 (commit)

Create a 2D table for `artist_id`, `track_count`, `avg_duration_ms`.

- Use the tracks list
- Verify the table shape
- Commit your results

Wrap-up

- Tables + pipelines make analytics predictable
- Keep columns aligned and verify shapes
- Next time: hash indexing for fast joins